

게임 수학

강의 5: 벡터의 응용

거리 계산, 충돌 감지, 투영, 평면

동명대학교 게임공학과
강영민

2026년 1학기

Contents

1	개요 — 왜 벡터 응용인가?	2
2	복습 — 점곱과 가위곱	2
3	점과 점 사이의 거리	4
4	점과 직선 사이의 거리	5
5	직선과 직선 사이의 거리	7
6	점과 평면 사이의 거리	9
7	충돌 감지 (Collision Detection)	10
8	레이 캐스팅 (Ray Casting)	11
9	최근접점 계산	12
10	Python 구현	12
11	연습 문제	15
12	요약	15

1 개요 — 왜 벡터 응용인가?

게임에서 수학을 가장 직접적으로 사용하는 장면 중 하나가 거리 계산과 충돌 감지이다. NPC의 시야 범위 확인, 탄환이 물체에 맞았는지 판정, 캐릭터가 벽에 너무 가까운지 체크 — 이 모든 것이 벡터 연산으로 구현된다.

이번 강의에서는 점, 선, 평면 사이의 거리를 벡터로 표현하고 계산하는 방법을 다룬다.

문제	핵심 연산
점과 점 사이의 거리	차 벡터의 크기
점과 직선 사이의 거리	정사영 후 거리 벡터
직선과 직선 사이의 거리	가위곱 법선 활용
점과 평면 사이의 거리	법선 벡터에 투영

2 복습 — 점곱과 가위곱

이번 강의의 모든 공식은 점곱(내적)과 가위곱(외적) 위에 세워진다. 핵심만 빠르게 복습하고 넘어가자.

2.1 점곱 (내적, Dot Product)

정의 — 점곱

두 벡터 $\mathbf{u} = (u_1, u_2, u_3)$, $\mathbf{v} = (v_1, v_2, v_3)$ 에 대해:

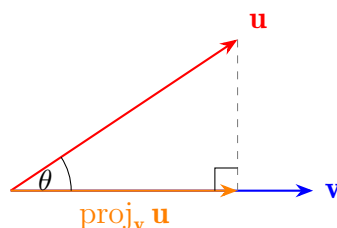
$$\mathbf{u} \cdot \mathbf{v} = u_1v_1 + u_2v_2 + u_3v_3 = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$$

결과는 스칼라이다.

핵심 성질.

- 교환 법칙: $\mathbf{u} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{u}$
- 직교 판정: $\mathbf{u} \perp \mathbf{v} \iff \mathbf{u} \cdot \mathbf{v} = 0$
- 크기와 관계: $\|\mathbf{u}\| = \sqrt{\mathbf{u} \cdot \mathbf{u}}$
- 두 벡터 사이의 각도: $\cos \theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$

투영 (Projection). $\mathbf{u}$ 를 $\mathbf{v}$ 방향으로 투영하면:



$$\text{스칼라 투영: } l = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{v}\|}, \quad \text{벡터 투영: } \text{proj}_{\mathbf{v}} \mathbf{u} = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{v}\|^2} \mathbf{v}$$

이번 강의에서의 활용

- 점-직선 거리: **AP**를 직선 방향에 투영 → 수선의 발
- 점-평면 거리: **AP**를 법선에 투영 → 부호 있는 거리
- 레이-평면 교점: $\hat{\mathbf{n}} \cdot \hat{\mathbf{d}}$ 로 평행 여부 판단
- 최근접점: 선분 매개변수 t 를 투영으로 계산

2.2 가위곱 (외적, Cross Product)

정의 — 가위곱 (3D 전용)

두 벡터 $\mathbf{u} = (u_1, u_2, u_3)$, $\mathbf{v} = (v_1, v_2, v_3)$ 에 대해:

$$\mathbf{u} \times \mathbf{v} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ u_1 & u_2 & u_3 \\ v_1 & v_2 & v_3 \end{vmatrix} = (u_2v_3 - u_3v_2, u_3v_1 - u_1v_3, u_1v_2 - u_2v_1)$$

결과는 벡터이며, 두 입력 벡터 모두에 수직이다. 방향은 오른손 법칙을 따른다.

핵심 성질.

- 반교환: $\mathbf{u} \times \mathbf{v} = -(\mathbf{v} \times \mathbf{u})$
- 크기 = 평행사변형 넓이: $\|\mathbf{u} \times \mathbf{v}\| = \|\mathbf{u}\| \|\mathbf{v}\| \sin \theta$
- 평행 판정: $\mathbf{u} \parallel \mathbf{v} \iff \mathbf{u} \times \mathbf{v} = \mathbf{0}$
- 자기 자신: $\mathbf{u} \times \mathbf{u} = \mathbf{0}$

이번 강의에서의 활용

- 점-직선 거리 (대안 공식): $d = \|(P - A) \times \mathbf{d}\| / \|\mathbf{d}\|$
- 꼬인 직선 거리: 공통 법선 $\mathbf{n} = \mathbf{d}_1 \times \mathbf{d}_2$ 방향으로 투영
- 평면의 법선: 평면 위 두 벡터의 가위곱 → 법선 벡터

2.3 점곱 vs 가위곱 비교

	점곱 $\mathbf{u} \cdot \mathbf{v}$	가위곱 $\mathbf{u} \times \mathbf{v}$
결과 타입	스칼라	벡터
기하적 의미	$\ \mathbf{u}\ \ \mathbf{v}\ \cos \theta$	수직 벡터, 크기 = $\ \mathbf{u}\ \ \mathbf{v}\ \sin \theta$
교환 법칙	$\mathbf{u} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{u}$	$\mathbf{u} \times \mathbf{v} = -(\mathbf{v} \times \mathbf{u})$
= 0 인 조건	직교 ($\theta = 90^\circ$)	평행 ($\theta = 0^\circ$ 또는 180°)
차원	n 차원 가능	3차원 전용

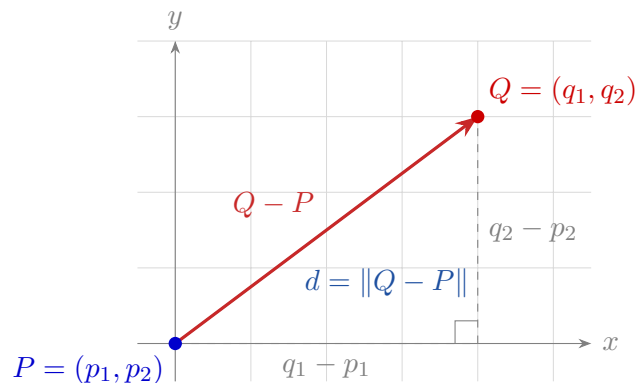
3 점과 점 사이의 거리

정의 — 두 점 사이의 거리

두 점 $P = (p_1, \dots, p_n), Q = (q_1, \dots, q_n) \in \mathbb{R}^n$ 에 대해 거리는:

$$d(P, Q) = \|Q - P\| = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

두 점 P 와 Q 가 주어졌을 때, 차 벡터 $Q - P$ 의 크기가 곧 거리이다.



$$d(P, Q) = \|Q - P\| = \sqrt{(q_1 - p_1)^2 + (q_2 - p_2)^2}$$

피타고라스 정리에서 직접 유도된다: x 축 방향 차이 $(q_1 - p_1)$ 와 y 축 방향 차이 $(q_2 - p_2)$ 를 직각삼각형의 두 변으로 놓으면, 빗변의 길이가 곧 d 이다. 3D에서도 같은 원리로 z 축 성분 $(q_3 - p_3)^2$ 이 추가된다.

거리의 대칭성. $d(P, Q) = d(Q, P)$ (순서 무관).

직접 비교 vs 제곱 비교. 게임에서 충돌 반경 내에 있는지 확인할 때는 $d \leq r$ 과 $d^2 \leq r^2$ 이 동치이므로, 제곱근 연산을 피하기 위해 거리 제곱을 비교하는 것이 효율적이다.

예제 1 — 공격 범위 판정

플레이어 위치 $P = (3, 0, 4)$, 적 위치 $E = (0, 0, 0)$, 공격 범위 $r = 6$. 적이 범위 내에 있는가?

$$d^2 = (3 - 0)^2 + (0 - 0)^2 + (4 - 0)^2 = 9 + 0 + 16 = 25$$

$$r^2 = 36$$

$$25 \leq 36 \checkmark \Rightarrow \text{적은 공격 범위 내에 있다. } (d = 5 \leq 6 = r)$$

4 점과 직선 사이의 거리

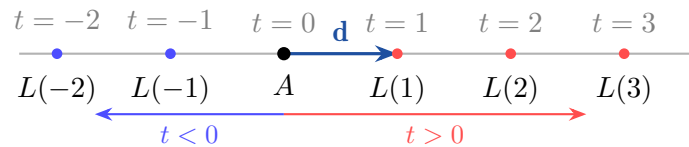
4.1 직선의 매개변수 방정식

정의 — 직선의 매개변수 표현

점 A 를 지나고 방향 $\mathbf{d}$ ($\|\mathbf{d}\| = 1$)인 직선 위의 점 $L(t)$:

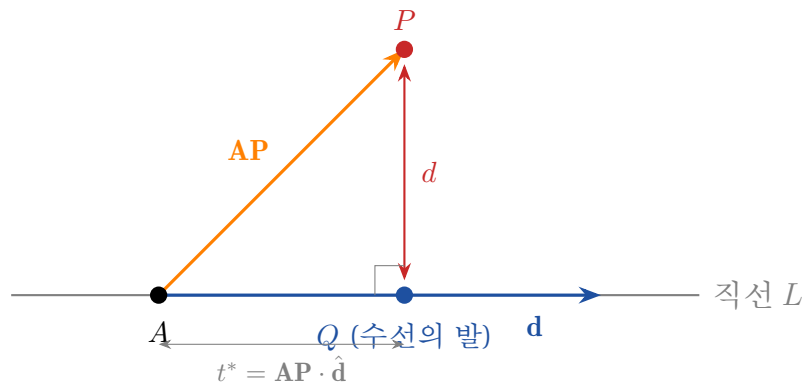
$$L(t) = A + t\mathbf{d}, \quad t \in \mathbb{R}$$

매개변수 t 를 바꾸면 직선 위의 모든 점을 얻을 수 있다. $t = 0$ 이면 시작점 A , $t > 0$ 이면 방향 $\mathbf{d}$ 쪽, $t < 0$ 이면 반대쪽이다.



4.2 점에서 직선까지의 거리

점 P 에서 직선 $L(t) = A + t\mathbf{d}$ 까지의 거리를 구하려면, 먼저 직선 위에서 P 와 가장 가까운 점 Q 를 찾아야 한다.



Q 는 $\mathbf{AP}$ 를 방향 $\mathbf{d}$ 에 정사영한 점이다:

$$t^* = (P - A) \cdot \mathbf{d}, \quad Q = A + t^*\mathbf{d}$$

거리 벡터: $\mathbf{PQ} = Q - P$, 거리: $d = \|\mathbf{Q} - \mathbf{P}\|$

정리 — 점과 직선 사이의 거리

단위 방향 벡터 $\hat{\mathbf{d}}$ 를 갖는 직선 $L = A + t\hat{\mathbf{d}}$ 에서 점 P 까지의 거리:

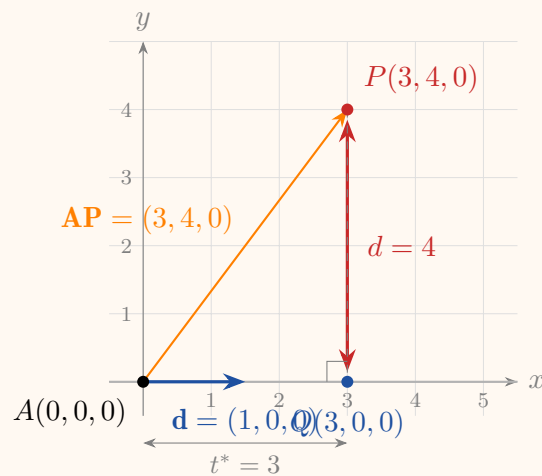
$$d = \left\| (P - A) - [(P - A) \cdot \hat{\mathbf{d}}]\hat{\mathbf{d}} \right\|$$

또는 가위곱을 이용하면 (3D):

$$d = \frac{\|(P - A) \times \mathbf{d}\|}{\|\mathbf{d}\|}$$

예제 2 — 점과 직선 거리

직선: $A = (0, 0, 0)$, 방향 $\mathbf{d} = (1, 0, 0)$. 점 $P = (3, 4, 0)$. 거리를 구하라.



$\hat{\mathbf{d}} = (1, 0, 0)$ (이미 단위 벡터)

$(P - A) = (3, 4, 0)$

정사영: $t^* = (3, 4, 0) \cdot (1, 0, 0) = 3$

수선의 발: $Q = (0, 0, 0) + 3(1, 0, 0) = (3, 0, 0)$

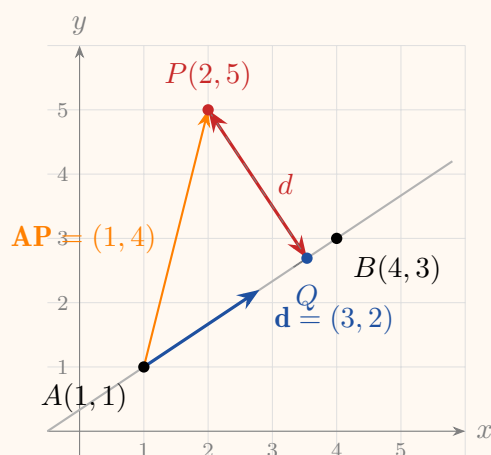
거리: $d = \|P - Q\| = \|(3, 4, 0) - (3, 0, 0)\| = \|(0, 4, 0)\| = 4$

검증 (가위곱 방법): $(P - A) \times \mathbf{d} = (3, 4, 0) \times (1, 0, 0) = (4 \cdot 0 - 0 \cdot 0, 0 \cdot 1 - 3 \cdot 0, 3 \cdot 0 - 4 \cdot 1) = (0, 0, -4)$

$d = \|(0, 0, -4)\| / \|(1, 0, 0)\| = 4 \checkmark$

예제 2b — 두 점으로 주어진 직선과 점의 거리

직선이 두 점 $A = (1, 1)$, $B = (4, 3)$ 을 지난다. 점 $P = (2, 5)$ 에서 이 직선까지의 거리를 구하라.



1단계: 방향 벡터 구하기. $\mathbf{d} = B - A = (4 - 1, 3 - 1) = (3, 2)$

2단계: 정사영 계산. $\mathbf{AP} = P - A = (2 - 1, 5 - 1) = (1, 4)$

$$t^* = \frac{\mathbf{AP} \cdot \mathbf{d}}{\|\mathbf{d}\|^2} = \frac{(1)(3) + (4)(2)}{3^2 + 2^2} = \frac{3+8}{13} = \frac{11}{13}$$

$$\text{3단계: 수선의 발. } Q = A + t^*\mathbf{d} = (1, 1) + \frac{11}{13}(3, 2) = \left(\frac{46}{13}, \frac{35}{13}\right)$$

$$\text{4단계: 거리 계산. } d = \|P - Q\| = \left\| \left(2 - \frac{46}{13}, 5 - \frac{35}{13}\right) \right\| = \left\| \left(-\frac{20}{13}, \frac{30}{13}\right) \right\| = \frac{1}{13}\sqrt{20^2 + 30^2} = \frac{\sqrt{1300}}{13} = \frac{10\sqrt{13}}{13} \approx 2.774$$

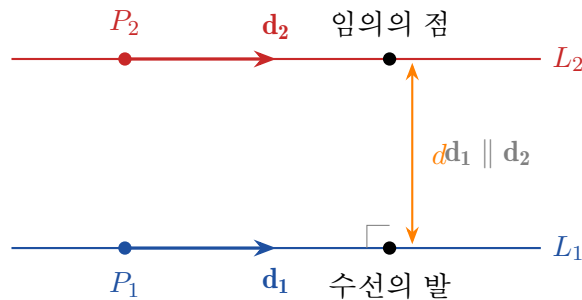
$$\text{검증 (가위곱 방법): } d = \frac{|(P - A) \times \mathbf{d}|}{\|\mathbf{d}\|} = \frac{|1 \cdot 2 - 4 \cdot 3|}{\sqrt{13}} = \frac{10}{\sqrt{13}} = \frac{10\sqrt{13}}{13} \checkmark$$

※ 두 점이 주어지면 $\mathbf{d} = B - A$ 로 방향 벡터를 만든 뒤, 같은 공식을 적용한다. 단, $\mathbf{d}$ 가 단위 벡터가 아니므로 $\|\mathbf{d}\|^2$ 으로 나눠야 한다.

5 직선과 직선 사이의 거리

5.1 평행한 두 직선

두 직선이 평행하면 ($\mathbf{d}_1 \parallel \mathbf{d}_2$), 한 직선 위의 임의의 점에서 다른 직선까지의 거리를 구하면 된다.



평행한 경우 어느 점을 택하든 거리가 같으므로, L_2 위의 아무 점(예: P_2)에서 L_1 까지의 점-직선 거리를 구하면 된다.

5.2 꼬인 위치의 두 직선 (3D)

두 직선이 꼬인 위치(skew lines)일 때, 두 직선에 동시에 수직인 벡터를 이용한다.

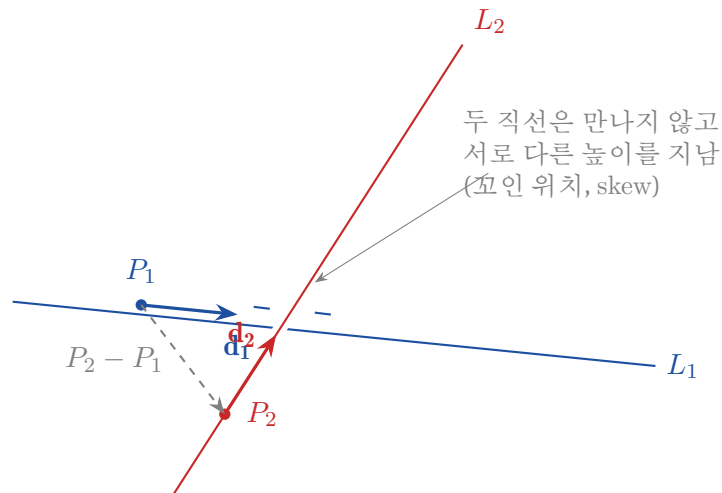
정리 — 꼬인 두 직선 사이의 거리

직선 $L_1 = P_1 + t\mathbf{d}_1$, $L_2 = P_2 + s\mathbf{d}_2$ 에 대해:

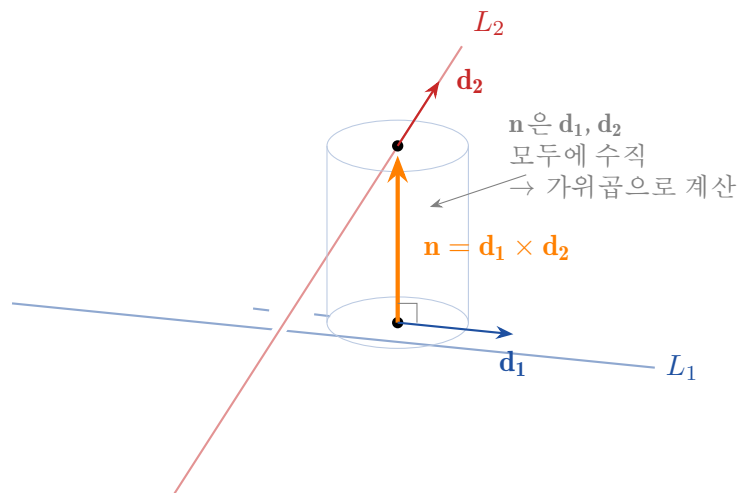
$$d = \frac{|(P_2 - P_1) \cdot (\mathbf{d}_1 \times \mathbf{d}_2)|}{\|\mathbf{d}_1 \times \mathbf{d}_2\|}$$

단, $\mathbf{d}_1 \times \mathbf{d}_2 \neq \mathbf{0}$ (평행하지 않을 때).

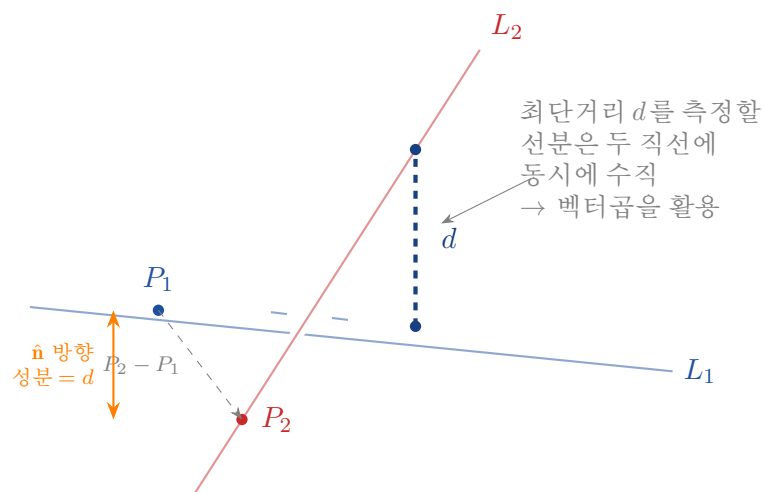
유도 과정. 1단계. 두 직선 L_1, L_2 가 꼬인 위치에 있다 — 만나지도, 평행하지도 않는다.



2단계. 공통 법선 벡터 $\mathbf{n} = \mathbf{d}_1 \times \mathbf{d}_2$ 은 두 직선 모두에 수직이다. 최단 거리 세그먼트는 이 법선 방향으로 놓여 있다.



3단계. $(P_2 - P_1)$ 을 법선 $\hat{\mathbf{n}}$ 방향으로 투영하면 최단 거리 d 를 얻는다. 최단 거리 세그먼트는 두 직선에 동시에 수직이다.



최종 공식:

$$d = |(P_2 - P_1) \cdot \hat{\mathbf{n}}| = \frac{|(P_2 - P_1) \cdot \mathbf{n}|}{\|\mathbf{n}\|}$$

예제 3 — 꼬인 두 직선 거리

$L_1: P_1 = (0, 0, 0), \mathbf{d}_1 = (1, 0, 0)$

$L_2: P_2 = (0, 1, 1), \mathbf{d}_2 = (0, 1, 0)$

두 직선 사이의 거리를 구하라.

$$\mathbf{n} = \mathbf{d}_1 \times \mathbf{d}_2 = (1, 0, 0) \times (0, 1, 0) = (0 \cdot 0 - 0 \cdot 1, 0 \cdot 0 - 1 \cdot 0, 1 \cdot 1 - 0 \cdot 0) = (0, 0, 1)$$

$$(P_2 - P_1) = (0, 1, 1)$$

$$d = \frac{|(0,1,1) \cdot (0,0,1)|}{\|(0,0,1)\|} = \frac{|0+0+1|}{1} = 1$$

6 점과 평면 사이의 거리

6.1 평면의 방정식

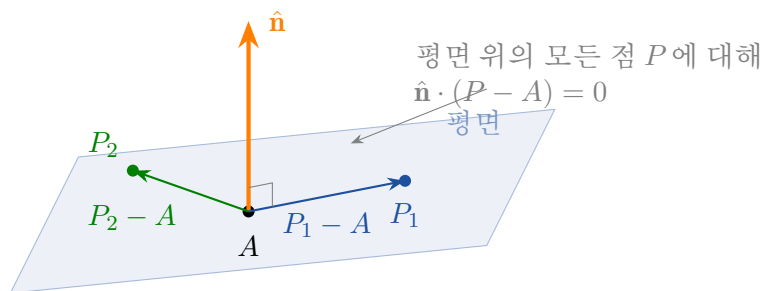
정의 — 평면의 방정식

단위 법선 벡터 $\hat{\mathbf{n}}$ 과 평면 위의 한 점 A 가 주어지면, 평면 위의 임의의 점 P 는 다음을 만족한다:

$$\hat{\mathbf{n}} \cdot (P - A) = 0$$

이를 일반형으로 쓰면 $ax + by + cz + d = 0$, 여기서 $(a, b, c) = \hat{\mathbf{n}}, d = -\hat{\mathbf{n}} \cdot A$.

법선 $\hat{\mathbf{n}}$ 과 점 A 가 평면을 유일하게 결정한다. 평면 위의 임의의 점 P 에서 A 로의 벡터 $(P - A)$ 는 반드시 법선에 수직이다.



6.2 점과 평면 사이의 부호 있는 거리

정리 — 점-평면 부호 있는 거리

법선 벡터 $\hat{\mathbf{n}}$ ($\|\hat{\mathbf{n}}\| = 1$), 평면 위의 점 A 로 정의된 평면에서 점 P 까지의 부호 있는 거리(signed distance):

$$d_s = \hat{\mathbf{n}} \cdot (P - A) = \hat{\mathbf{n}} \cdot P - \hat{\mathbf{n}} \cdot A$$

- $d_s > 0$: P 는 법선이 향하는 쪽 (앞면)
- $d_s = 0$: P 는 평면 위

- $d_s < 0$: P 는 법선 반대쪽 (뒷면)

부호 있는 거리는 다음에 활용된다:

- 카메라 클리핑 평면(near/far plane) 안에 물체가 있는지 판단
- BSP 트리(Binary Space Partitioning) 구성
- 충돌 응답에서 물체를 평면 위로 밀어내기

예제 4 — 점과 평면 거리

평면: 법선 $\hat{\mathbf{n}} = (0, 1, 0)$, 점 $A = (0, 3, 0)$ ($y=3$ 평면). 점 $P = (5, 7, 2)$ 에서 평면까지의 거리를 구하라.

$$\begin{aligned} d_s &= \hat{\mathbf{n}} \cdot (P - A) = (0, 1, 0) \cdot (5, 7, 2) - (0, 1, 0) \cdot (0, 3, 0) \\ &= (0 \cdot 5 + 1 \cdot 7 + 0 \cdot 2) - (0 \cdot 0 + 1 \cdot 3 + 0 \cdot 0) = 7 - 3 = 4 \end{aligned}$$

P 는 평면 앞면에서 거리 4 떨어져 있다.

7 충돌 감지 (Collision Detection)

7.1 구와 구의 충돌

두 구의 충돌은 가장 단순한 경우로, 단순히 중심 간 거리와 반지름 합을 비교한다.

정리 — 구-구 충돌

중심 C_1 , 반지름 r_1 인 구와 중심 C_2 , 반지름 r_2 인 구의 충돌 조건:

$$\|C_2 - C_1\|^2 \leq (r_1 + r_2)^2$$

(제곱근 연산 회피를 위해 양변을 제곱하여 비교)

엡실론(epsilon) 비교

부동소수점 연산에서는 정확히 같다($= 0$)는 조건을 $|x| < \varepsilon$ 으로 대체한다. 보통 $\varepsilon = 10^{-6}$ 또는 10^{-9} 정도를 사용.

7.2 구와 평면의 충돌

정리 — 구-평면 충돌

중심 C , 반지름 r 인 구와 법선 $\hat{\mathbf{n}}$, 점 A 를 지나는 평면의 충돌 조건:

$$|\hat{\mathbf{n}} \cdot (C - A)| \leq r$$

즉, 구 중심과 평면 사이의 거리가 반지름 이하면 충돌.

7.3 점과 구의 충돌 (점이 구 안에 있는지)

$$\|P - C\|^2 \leq r^2$$

7.4 AABB (Axis-Aligned Bounding Box)

정의 — AABB

각 축에 정렬된 직육면체 충돌 영역. 최소점 $\mathbf{min} = (x_{min}, y_{min}, z_{min})$, 최대점 $\mathbf{max} = (x_{max}, y_{max}, z_{max})$.

두 AABB가 겹치는 조건 (모든 축에서 겹쳐야 함):

$$A_{min_x} \leq B_{max_x} \wedge B_{min_x} \leq A_{max_x} \wedge A_{min_y} \leq B_{max_y} \wedge \dots$$

8 레이 캐스팅 (Ray Casting)

레이 캐스팅은 게임에서 가장 자주 사용되는 기법 중 하나이다. 마우스 클릭으로 오브젝트를 선택하거나, 총알이 물체에 맞는지 계산하거나, 시야(line of sight)를 확인할 때 쓰인다.

정의 — 레이(Ray)

시작점 O 와 방향 $\hat{\mathbf{d}}$ 로 정의된 반직선:

$$R(t) = O + t\hat{\mathbf{d}}, \quad t \geq 0$$

8.1 레이와 평면의 교점

정리 — 레이-평면 교점

레이 $R(t) = O + t\hat{\mathbf{d}}$ 와 법선 $\hat{\mathbf{n}}$, 점 A 를 지나는 평면의 교점:

$$t = \frac{\hat{\mathbf{n}} \cdot (A - O)}{\hat{\mathbf{n}} \cdot \hat{\mathbf{d}}}$$

$\hat{\mathbf{n}} \cdot \hat{\mathbf{d}} = 0$ 이면 레이와 평면이 평행 (교점 없음). $t < 0$ 이면 레이 뒤쪽 (교점 없음).

유도. 교점 $P = O + t\hat{\mathbf{d}}$ 는 평면 위에 있으므로 $\hat{\mathbf{n}} \cdot (P - A) = 0$:

$$\hat{\mathbf{n}} \cdot (O + t\hat{\mathbf{d}} - A) = 0 \quad \Rightarrow \quad t = \frac{\hat{\mathbf{n}} \cdot (A - O)}{\hat{\mathbf{n}} \cdot \hat{\mathbf{d}}}$$

8.2 레이와 구의 교점

정리 — 레이-구 교점

레이 $R(t) = O + t\hat{\mathbf{d}}$ 와 중심 C , 반지름 r 인 구의 교점:

$$\mathbf{oc} = O - C, \quad a = \hat{\mathbf{d}} \cdot \hat{\mathbf{d}} = 1, \quad b = 2(\mathbf{oc} \cdot \hat{\mathbf{d}}), \quad c = \mathbf{oc} \cdot \mathbf{oc} - r^2$$

판별식 $\Delta = b^2 - 4c$:

- $\Delta < 0$: 교점 없음
- $\Delta = 0$: 접선 (1개의 교점)
- $\Delta > 0$: 2개의 교점, $t = \frac{-b \pm \sqrt{\Delta}}{2}$

9 최근접점 계산

9.1 선분 위의 최근접점

무한 직선이 아니라 유한 선분 AB (A 에서 B 까지)에서 점 P 의 최근접점을 구할 때는 t 를 $[0, 1]$ 범위로 클램프(clamp)한다:

$$t = \text{clamp}\left(\frac{(P - A) \cdot (B - A)}{\|B - A\|^2}, 0, 1\right)$$
$$Q = A + t(B - A)$$

$t = 0$ 이면 A 가 최근접, $t = 1$ 이면 B 가 최근접, $0 < t < 1$ 이면 선분 내부의 점이 최근접.

예제 5 — 선분 최근접점

선분 $A = (0, 0, 0)$, $B = (4, 0, 0)$. 점 $P = (5, 3, 0)$ 의 최근접점을 구하라.

$$(P - A) \cdot (B - A) = (5, 3, 0) \cdot (4, 0, 0) = 20$$

$$\|B - A\|^2 = 16$$

$$t = 20/16 = 1.25 \xrightarrow{\text{clamp}} 1$$

$$Q = A + 1 \cdot (B - A) = B = (4, 0, 0)$$

$$\Rightarrow \text{최근접점은 } B = (4, 0, 0), \text{ 거리} = \|P - B\| = \|(1, 3, 0)\| = \sqrt{10}$$

10 Python 구현

실습

```
1 import numpy as np
2
3 # =====
4 # 1. 점과점사이의거리제공 (비교)
```

```

5 # =====
6 def dist_sq(P, Q):
7     """거리 제곱반환총돌 ( 판정에유리 )"""
8     diff = Q - P
9     return np.dot(diff, diff)
10
11 def dist(P, Q):
12     return np.sqrt(dist_sq(P, Q))
13
14 P = np.array([3.0, 0.0, 4.0])
15 E = np.array([0.0, 0.0, 0.0])
16 r = 6.0
17 print("거리:", dist(P, E)) # 5.0
18 print("범위 내?", dist_sq(P, E) <= r**2) # True
19
20 # =====
21 # 2. 점과직선사이의거리
22 # =====
23 def point_to_line_dist(P, A, d):
24     """에서P A+t*d 직선까지의거리 (는d 방향벡터, 단위벡터아니어도
        됨 )"""
25     d_hat = d / np.linalg.norm(d)
26     AP = P - A
27     t = np.dot(AP, d_hat)
28     Q = A + t * d_hat # 수선의발
29     return np.linalg.norm(P - Q), Q
30
31 A = np.array([0.0, 0.0, 0.0])
32 d = np.array([1.0, 0.0, 0.0])
33 P = np.array([3.0, 4.0, 0.0])
34 dist_val, foot = point_to_line_dist(P, A, d)
35 print(f"점직선- 거리: {dist_val:.4f}") # 4.0
36 print(f"수선의 발: {foot}") # [3. 0. 0.]
37
38 # =====
39 # 3. 꼬인두직선사이의거리
40 # =====
41 def skew_lines_dist(P1, d1, P2, d2):
42     n = np.cross(d1, d2)
43     n_len = np.linalg.norm(n)
44     if n_len < 1e-9:
45         # 평행한직선 : 한점에서다른직선까지거리
46         return point_to_line_dist(P1, P2, d2)[0]
47     return abs(np.dot(P2 - P1, n)) / n_len
48
49 P1 = np.array([0.0, 0.0, 0.0])
50 d1 = np.array([1.0, 0.0, 0.0])
51 P2 = np.array([0.0, 1.0, 1.0])
52 d2 = np.array([0.0, 1.0, 0.0])
53 print(f"꼬인 직선거

```

```

리 : {skew_lines_dist(P1,d1,P2,d2):.4f}") # 1.0
54
55 # =====
56 # 4. 점과평면사이의부호있는거리
57 # =====
58 def signed_dist_point_plane(P, n_hat, A):
59     """부호 있는거리 . 양수 = 법선방향 , 음수 = 반대"""
60     return np.dot(n_hat, P - A)
61
62 n = np.array([0.0, 1.0, 0.0])
63 A = np.array([0.0, 3.0, 0.0])
64 P = np.array([5.0, 7.0, 2.0])
65 sd = signed_dist_point_plane(P, n, A)
66 print(f"부호 있는거리 : {sd:.4f}") # 4.0
67
68 # =====
69 # 5. 구구- 충돌감지
70 # =====
71 def sphere_sphere_collision(C1, r1, C2, r2):
72     return dist_sq(C1, C2) <= (r1 + r2)**2
73
74 # =====
75 # 6. 레이구- 교점
76 # =====
77 def ray_sphere_intersect(O, d_hat, C, r):
78     oc = O - C
79     b = 2 * np.dot(oc, d_hat)
80     c = np.dot(oc, oc) - r**2
81     discriminant = b**2 - 4*c
82     if discriminant < 0:
83         return None, None
84     sqrt_disc = np.sqrt(discriminant)
85     t1 = (-b - sqrt_disc) / 2
86     t2 = (-b + sqrt_disc) / 2
87     return t1, t2
88
89 O = np.array([0.0, 0.0, -5.0])
90 d = np.array([0.0, 0.0, 1.0])
91 C = np.array([0.0, 0.0, 0.0])
92 r = 2.0
93 t1, t2 = ray_sphere_intersect(O, d, C, r)
94 print(f"레이구- 교점 t: {t1:.2f}, {t2:.2f}") # 3.0, 7.0
95
96 # =====
97 # 7. 선분위의최근접점
98 # =====
99 def closest_point_on_segment(P, A, B):
100     AB = B - A
101     t = np.dot(P - A, AB) / np.dot(AB, AB)
102     t = np.clip(t, 0.0, 1.0)

```

```

103     return A + t * AB
104
105 P = np.array([5.0, 3.0, 0.0])
106 A = np.array([0.0, 0.0, 0.0])
107 B = np.array([4.0, 0.0, 0.0])
108 Q = closest_point_on_segment(P, A, B)
109 print(f"최근접점: {Q}, 거리: {np.linalg.norm(P-Q):.4f}")

```

11 연습 문제

연습 문제

1. 점 $P = (1, 2, 3)$ 에서 직선 $L(t) = (0, 0, 0) + t(1, 1, 0)/\sqrt{2}$ 까지의 거리를 구하라.
2. 반지름 3인 구(중심 원점)에 레이 $O = (5, 0, 0)$, $\hat{\mathbf{d}} = (-1, 0, 0)$ 이 맞는 경우 교점 좌표를 구하라.
3. 평면 $2x + y - 2z = 6$ 과 점 $P = (1, 1, 1)$ 사이의 거리를 구하라. (힌트: 법선 벡터를 먼저 정규화하라)
4. AABB 최소점 $(0, 0, 0)$, 최대점 $(5, 5, 5)$ 와 중심 $(6, 3, 3)$, 반지름 2인 구의 충돌 여부를 판정하라. (힌트: AABB 위의 최근접점을 먼저 구하라)
5. 직선 $L_1: P_1 = (1, 0, 0), \mathbf{d}_1 = (0, 1, 0)$ 와
직선 $L_2: P_2 = (0, 0, 1), \mathbf{d}_2 = (1, 0, 0)$
의 거리를 구하라.
6. Python으로 위 문제들을 구현하고 결과를 출력하라.

12 요약

핵심 정리

1. 점-직선 거리: $\mathbf{AP}$ 를 방향 벡터에 투영한 후 남은 수직 성분의 크기.
2. 꼬인 직선 거리: 공통 법선 $\mathbf{d}_1 \times \mathbf{d}_2$ 에 투영하여 계산.
3. 점-평면 거리: 법선 벡터 방향 투영 $\hat{\mathbf{n}} \cdot (P - A)$; 부호로 어느 쪽인지 알 수 있다.
4. 충돌 감지: 제곱 거리 비교로 제곱근을 피하면 효율적이다.
5. 레이 캐스팅: 이차방정식으로 교점 t 를 구하고, $t \geq 0$ 인 가장 작은 값이 첫 교점.
6. 선분 최근접점: 무한 직선의 t 를 $[0, 1]$ 로 클램프한다.